AI-Powered Fake News Detector Design Document

Technical Design, Workflow, and Architecture Overview

1. Overview

This application is an AI-powered fake news detection system built using:

- Streamlit for the user interface
- LangGraph for workflow orchestration
- Tavily for web evidence retrieval
- Ollama for local language model inference

The system accepts a user-submitted news claim, gathers supporting evidence from the web, analyzes it using a local LLM, and returns a verdict such as:

- **REAL**
- **FAKE**
- **MISLEADING**
- **UNVERIFIED**

2. Technology Stack

The application uses the following components:

Frontend

- Streamlit Used for:
    - Chat interface
    - Sidebar configuration

    - Response display
    - Session state handling

Workflow Engine

- LangGraph Used to:

    - Create the ReAct agent
    - Manage reasoning flow
    - Invoke tools dynamically

Search Tool

- Tavily Used to:
    - Search the web
    - Fetch evidence sources
    - Provide search summaries

LLM Runtime

- Ollama Used to:
    - Run local models
    - Reason over evidence
    - Produce structured verdicts

3. Architecture Diagram

```
User → Streamlit UI → LangGraph Agent
                         ↙        ↘
                  Tavily Search   Ollama Model
                         ↓
                  Fact Check Result
                         ↓
                    Streamlit UI
                         ↓
                        User
```

This architecture illustrates the interaction between the user interface, the agent, the web search tool, and the language model.

4. Workflow Description

The system processes requests in the following order:

Step 1: User Input

The user enters a claim into the Streamlit chat interface.

Example:

“NASA confirms aliens found on Mars”

Step 2: Agent Receives Input

The LangGraph agent receives the claim as input.

The agent determines:

- whether web search is needed
- what evidence to gather
- how to structure the final answer

Step 3: Tavily Search Retrieves Evidence

Tavily searches the web for evidence related to the claim.

It returns:

- search results
- evidence snippets
- source links

Step 4: Ollama Analyzes the Evidence

The selected Ollama model evaluates:

- evidence credibility
- source consistency
- truthfulness of the claim

It then generates:

- Verdict
- Explanation
- Sources

Step 5: Final Output Formatting

The application formats the result into:

- **Verdict**
- **Explanation**
- **Sources**

This output is rendered in the Streamlit UI.

5. User Interface Layer

The Streamlit interface provides:

Chat Input

Allows the user to enter a claim for analysis.

Model Selection

Lets the user select an Ollama model:

- `llama3.2:3b`

- `gemma2:2b`
- `phi3:3.8b`
- `llama3.1:8b`

Search Results Slider

Lets the user control how many Tavily search results to fetch.

Response Timing

Displays the total time taken to analyze the claim.

6. Agent Layer

The LangGraph ReAct Agent acts as the central decision-making layer.

Responsibilities:

- Receive user input
- Call Tavily Search
- Send evidence to Ollama
- Generate the structured response

This is the intelligence orchestration layer of the application.

7. Search Layer

The Tavily Search tool is responsible for:

- searching the internet for relevant evidence
- retrieving credible sources
- summarizing findings for the LLM

This gives the LLM access to up-to-date external information.

8. LLM Layer

The Ollama model performs the reasoning task.

It receives:

- the user claim
- evidence from Tavily
- the system prompt

It returns:

- **Verdict**
- **Explanation**
- **Sources**

The model can be configured based on performance and accuracy requirements.

9. Output Rendering Logic

The output is formatted into three sections:

Verdict

Possible verdicts:

- REAL
- FAKE
- MISLEADING
- UNVERIFIED

Each verdict is displayed with an icon:

- **REAL** → ✅
- **FAKE** → ❌
- **MISLEADING** → ⚠️
- **UNVERIFIED** → ❓

Explanation

A concise evidence-based summary explaining the verdict.

Sources

A list of important sources used in the decision.

10. Session Management

The application stores state using Streamlit session state:

Stored State

- Selected model
- Conversation history
- Agent state

This preserves user interaction across reruns.

11. Error Handling

The application handles runtime failures gracefully.

Examples:

- Search API errors
- Model errors
- Formatting errors

Instead of crashing, the app displays an error message to the user.

12. Performance Monitoring

The application measures:

- total response time per request

Example:

```
⏱️ Took 2.4 seconds
```

This helps monitor application performance.

13. Future Enhancements

Potential improvements:

- Confidence score for verdicts
- Source credibility ranking
- Persistent database storage
- Analytics dashboard
- Multi-model comparison mode
- Better verdict cards

14. Conclusion

This application combines:

- Streamlit
- LangGraph
- Tavily
- Ollama

to build an AI-powered fake news detection workflow.

The architecture is modular, scalable, and capable of producing explainable verdicts based on web evidence and LLM reasoning.